
Deep Reinforcement Learning for Compiler Optimization

Pass Ordering in LLVM

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 The phase ordering problem—determining the optimal sequence of compiler opti-
2 mization passes—is NP-hard and has resisted efficient exact solutions for decades.
3 We investigate how far traditional deep reinforcement learning (DRL) can be
4 pushed on this problem without relying on large language models or hand-crafted
5 heuristics, using the CompilerGym Cummins et al. [2022] LLVM environment as
6 our testbed.

7 Our work makes three contributions. First, through a systematic ablation of six
8 state representations, we identify *pass_sequence*—encoding the ordered history
9 of the k most recent optimization passes as concatenated one-hot vectors—as the
10 strongest representation in our evaluated design space, outperforming IR-feature-
11 based (Autophase) and count-based alternatives by a large margin. Second, we
12 develop a scalable training infrastructure with truncation-aware Generalized Ad-
13 vantage Estimation Schulman et al. [2015], 8-worker parallel rollout collection
14 ($7.6\times$ speedup), and per-suite reward normalization for stable multi-suite joint
15 training. Third, an inference-time ensemble of seven independently trained poli-
16 cies with best-of-8 stochastic rollouts achieves mean reward 1.050 on cbench-v1
17 and beats -0z on 20 out of 23 programs—doubling the count compared to our
18 recurrent PPO baseline (10/23). A single-model best-of-16 policy reaches mean
19 reward 1.060 and beats -0z on 21 out of 23 programs using only 16 rollouts,
20 outperforming a random best-of-56 baseline (0.992 mean reward, 5/23 programs)
21 under a controlled compute budget. Cross-suite evaluation across six additional
22 benchmark suites demonstrates zero-shot generalization.

1 Introduction

24 The phase ordering problem is a fundamental challenge in compiler optimization: it asks for the
25 optimal ordering of compilation passes such that the resulting low-level code achieves the best per-
26 formance or the smallest code size. Different pass orderings can produce substantially different ma-
27 chine code, making the choice of optimization schedule particularly consequential in performance-
28 critical settings. Despite decades of research, the problem remains NP-hard in the general case, and
29 no efficient algorithm is known to find globally optimal pass sequences.

30 Classical solutions rely on fixed schedules (-02, -03, -0z) that represent manually tuned compro-
31 mises over a broad class of programs. While effective in practice, these schedules cannot adapt to the
32 structure of individual programs, leaving program-specific optimization opportunities unexploited.
33 Iterative compilation methods search the pass-order space exhaustively but scale poorly.

34 The recent rise of reinforcement learning (RL) offers a natural fit: an agent can interact with the com-
35 piler step by step, observe the evolving intermediate representation, and learn a policy that selects
36 passes sequentially. To facilitate such approaches, CompilerGym Cummins et al. [2022] provides
37 a standardized gym-compatible interface to LLVM optimization with reproducible benchmarks and
38 objective reward signals.

39 Despite this promise, recent work has moved toward large language model (LLM)-based ap-
40 proaches Pan et al. [2026], which require orders of magnitude more compute and sacrifice the sample
41 efficiency of actor-critic methods. This paper instead investigates how far *traditional* DRL can be
42 pushed, focusing on two questions: (1) *what is the best way to represent the compiler state for a*
43 *DRL agent?* and (2) *how can inference-time diversity be exploited to further improve performance*
44 *without additional training cost?*

45 We make the following contributions:

- 46 • **State representation study.** A controlled ablation of six representations shows that
47 `pass_sequence` ($k=10$)—encoding the ordered history of recent passes—is the dominant
48 design choice (avg reward $0.510 \rightarrow 0.937$ over normalized Autophase).
- 49 • **Scalable training infrastructure.** Truncation-aware GAE, 8-worker parallel rollouts, and
50 per-suite Welford normalization enable stable multi-suite training across suites with up to
51 $42\times$ reward variance ratio.
- 52 • **Inference-time ensemble.** Seven independently trained policies with bo8 rollouts achieve
53 avg 1.050 on cbench-v1 ($20/23 \geq -0z$), generalizing zero-shot to six additional suites.
- 54 • **Compute-budget comparison.** MLP-Seq-Full (bo16) (16 rollouts, avg 1.060,
55 $21/23 \geq -0z$) outperforms random best-of-56 (56 rollouts, avg 0.992, $5/23$) on all 23 bench-
56 marks.

57 2 Related Work

58 2.1 Compiler Optimization as a Sequential Decision Problem

59 Early iterative-compilation systems such as MILEPOST Fursin et al. [2011] treated pass ordering
60 as a supervised or genetic-search problem with offline models that cannot adapt at inference time.
61 CompilerGym Cummins et al. [2022] formalizes the problem as a Markov Decision Process, provid-
62 ing a gym-compatible interface with objective reward signals. Our work builds directly on this
63 framework.

64 2.2 Reinforcement Learning for Compiler Optimization

65 Huang et al. Huang et al. [2019] introduced Autophase, a 56-dimensional IR feature vector, and
66 showed that a simple RL agent trained on it can surpass fixed optimization levels on individual pro-
67 grams. Cummins et al. Cummins et al. [2021] proposed graph-based IR representations (ProGraML)
68 for learning program structure at higher training cost. Compiler-R1 Pan et al. [2026] applies LLMs
69 with reinforcement fine-tuning, achieving strong results at orders-of-magnitude higher compute. Our
70 work establishes a strong traditional-DRL baseline and a systematic understanding of what represen-
71 tations and inference strategies matter most.

72 2.3 PPO, ICM, and Ensemble Methods

73 PPO Schulman et al. [2017] is applied to combinatorial action spaces including chip place-
74 ment Mirhoseini et al. [2021]; we adopt MLP (ablation) and recurrent GRU (ensemble) variants.
75 ICM Pathak et al. [2017] adds a prediction-error exploration bonus; in our setup the signal decays to
76 negligible magnitude by mid-training. Ensemble methods Lakshminarayanan et al. [2017], Osband
77 et al. [2016] exploit policy diversity; each run converges to a different attractor, and the per-program
78 max recovers the best-fitting policy at inference time.

79 3 Methodology

80 Our investigation proceeds in two phases. **Phase 1** (Section 4.2–4.5) conducts controlled ablations
 81 using the full 124-pass action space and a feed-forward MLP to identify the optimal state represen-
 82 tation. **Phase 2** (Section 4.6–4.7) scales up with a recurrent GRU policy, curated 30-pass action set,
 83 parallel rollouts, and an inference-time ensemble. The key insight from Phase 1—that sequential
 84 pass history is the most informative signal—motivates the history-augmented state in Phase 2.

85 3.1 MDP Formulation

86 **State.** We evaluate two families of state representations (detailed in Section 4.2).

87 *Pass-sequence encoding* (`pass_sequence`, k): the ordered history of the k most recent passes,
 88 encoded as a concatenation of k one-hot vectors over the 124-pass action space, yielding $k \times 124$
 89 dimensions. Episodes shorter than k steps are zero-padded at the front. This is the best-performing
 90 representation identified by our ablation ($k=10$, 1240-dim).

91 *Autophase with per-action usage counts*: the 56-dimensional Autophase vector Huang et al. [2019],
 92 per-feature online normalized via a Welford estimator $\hat{\phi}_i = (\phi_i - \mu_i)/(\sigma_i + \epsilon)$, concatenated with
 93 a vector recording how many times each pass has been applied this episode. This representation is
 94 used in the recurrent ensemble model (Phase 2), where the GRU hidden state additionally models
 95 pass-interaction history.

96 **Action.** Phase 1 uses the full 124-pass LLVM action space. Phase 2 uses a curated 30-pass subset
 97 (PASSES_V4) derived from the -Oz pass schedule, which reduces the search space while retaining
 98 the most effective passes.

99 **Reward.** Both phases use the `IrInstructionCount0z` reward, which is implemented as a *per-*
 100 *step* incremental signal:

$$r_t = \frac{\text{IC}(P_{t-1}) - \text{IC}(P_t)}{\max(\text{IC}_{-O0}(P) - \text{IC}_{-Oz}(P), 1)}, \quad (1)$$

101 where $\text{IC}(P_t)$ is the IR instruction count after applying pass a_t , and the denominator normalizes by
 102 the improvement that -Oz achieves over -O0 (the unoptimized baseline). The first step initializes
 103 $\text{IC}(P_0) = \text{IC}_{-O0}(P)$.

104 Because Eq. (1) is a telescoping sum, the cumulative episode reward simplifies to:

$$R = \sum_{t=1}^T r_t = \frac{\text{IC}_{-O0}(P) - \text{IC}(P_T)}{\text{IC}_{-O0}(P) - \text{IC}_{-Oz}(P)}, \quad (2)$$

105 where P_T is the final program after the agent’s pass sequence. $R = 1.0$ means the agent matches
 106 -Oz; $R > 1.0$ means it surpasses -Oz; $R < 0$ means the agent performs worse than the unoptimized
 107 -O0 baseline.

108 **POMDP structure.** Compiler pass ordering is a POMDP: Autophase provides only a partial ob-
 109 servation, causing state aliasing—identical observations may require different actions depending on
 110 transformation history. `pass_sequence` approximates the belief state $b_t = P(s_t | o_1, a_1, \dots, o_t)$ by
 111 supplying the last k actions as an explicit history signal, explaining the gap between `autophase_norm`
 112 (avg 0.510) and `pass_sequence` (avg 0.937).

113 **Episode length.** $T = 45$ steps maximum (TimeLimit wrapper). Extending to $T = 90$ degraded
 114 performance, confirming that the binding constraint is the action set, not the time budget.

115 3.2 Phase 1: Feed-Forward MLP Policy

116 For ablation studies, we use a two-layer MLP (256×256 , ReLU) as the actor-critic backbone. The
 117 MLP takes the state representation directly as input and outputs a logit vector over the action space
 118 and a scalar value estimate. This non-recurrent architecture is deliberately simple to isolate the effect
 119 of the state representation.

120 3.3 Phase 2: Recurrent GRU Policy with ICM

121 For ensemble training, we use a recurrent actor-critic with a GRU backbone. The state s'_t is encoded
 122 by an MLP with LayerNorm and GELU activations: $\phi_t = f_{\theta}^{\text{enc}}(s'_t)$. The GRU conditions on both
 123 the latent state and the previous action, $h_t = \text{GRU}([\phi_t \| a_{t-1}^{\text{onehot}}], h_{t-1})$, enabling the policy to
 124 reason about transformation history in this POMDP setting. The shared h_t feeds actor and critic
 125 heads:

$$\pi_{\theta}(a_t | h_t) = \text{softmax}(f_{\theta}^{\pi}(h_t)), \quad V_{\theta}(s_t) = f_{\theta}^v(h_t). \quad (3)$$

126 **Intrinsic Curiosity Module (ICM).** To encourage early-training exploration, we augment the
 127 extrinsic reward with an ICM Pathak et al. [2017] bonus. A forward model predicts $\hat{\phi}_{t+1} =$
 128 $g_{\psi}^{\text{fwd}}(\phi_t, a_t)$ and produces intrinsic reward $r_t^{\text{int}} = \frac{1}{2} \|\hat{\phi}_{t+1} - \phi_{t+1}\|^2$. An inverse model
 129 $g_{\psi}^{\text{inv}}(\phi_t, \phi_{t+1})$ is trained via cross-entropy loss to preserve action-predictive information in ϕ_t . The
 130 total reward is $r'_t = r_t^{\text{ext}} + \beta r_t^{\text{int}}$; empirically, the ICM signal decays to negligible magnitude by
 131 mid-training.

132 3.4 PPO Training and Truncation-Aware GAE

133 We train the agent using PPO Schulman et al. [2017] with the clipped surrogate objective:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t[\min(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A_t)], \quad (4)$$

134 where $\rho_t = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ is the probability ratio and A_t is the GAE advantage estimate.
 135 Standard GAE Schulman et al. [2015] sets the bootstrap value to zero at terminal states. When
 136 an episode ends by time-limit truncation rather than a true terminal transition, the correct target is
 137 $V(s_T)$, not zero. We maintain a truncation buffer that distinguishes truncated from truly terminal
 138 transitions, bootstrapping $V(s_T)$ at truncation boundaries. This eliminates a systematic underesti-
 139 mation of long-horizon credit.

140 3.5 Parallel Rollout Collection

141 We use $W=8$ parallel workers in a synchronous actor-learner architecture. Each worker maintains an
 142 independent CompilerGym gRPC service; the main process broadcasts weights and normalization
 143 statistics before each collection phase. The first transition per worker is flagged as a new episode
 144 boundary to reset the GRU hidden state. This reduces training from ≈ 3 hours to 47 minutes per
 145 300k steps ($7.6\times$ speedup).

146 3.6 Inference-Time Ensemble

147 Given N checkpoints $\{\pi_1, \dots, \pi_N\}$ and rollout budget k per model, the ensemble selects for each
 148 program p :

$$\hat{R}(p) = \max_{i \in [N], j \in [k]} R(\tau_{i,j}(p)). \quad (5)$$

149 We set $k=8$ (bo8) and $N \in \{5, 6, 7\}$. Each run—with distinct learning rate, entropy coefficient,
 150 or state-feature configuration—converges to a different attractor Lakshminarayanan et al. [2017],
 151 Osband et al. [2016]; the per-program maximum recovers the best-fitting attractor at inference time
 152 with no training cost.

153 3.7 Per-Suite Reward Normalization

154 Joint training causes per-step rewards to span $[-30, +5]$, leading to value-loss divergence at step
 155 80k. We apply a per-suite Welford z-score: $\tilde{r}_t^{(s)} = \text{clip}((r_t^{(s)} - \mu^{(s)}) / (\sigma^{(s)} + \epsilon), -5, +5)$. This
 156 keeps value loss in $[0.2, 0.8]$ across 300k steps despite a $42\times$ per-suite reward variance ratio (cbench
 157 $\sigma=0.11$, opencv $\sigma=4.44$).

4 Experiments

4.1 Setup

Environment. We use CompilerGym Cummins et al. [2022] llvm-v0 with the IrInstructionCountOz reward. cbench-v1 is a standard compiler benchmark suite comprising 23 C programs spanning embedded workloads (cryptography, image processing, data compression, string search, and networking). All models are **trained exclusively on cbench-v1** and evaluated both in-distribution (cbench-v1) and zero-shot on six held-out suites: mibench-v1, blas-v0, npb-v0, chstone-v0, opencv-v0, and anghabench-v1, covering linear algebra kernels, HPC benchmarks, and large open-source C codebases.

Algorithm. PPO Schulman et al. [2017]: clip $\varepsilon=0.2$, $\gamma=0.99$, GAE $\lambda=0.95$ Schulman et al. [2015], $n_steps=512$, entropy_coef= 0.01, value_coef= 0.5, lr= 3×10^{-4} .

Hardware. Phase 1: NVIDIA RTX 5070 Ti (500 ep/benchmark). Phase 2: Apple Mac mini M4 Pro, 24 GB, 8 CPU workers, ≈ 47 min/300k steps.

Baselines. **Random** (greedy); **Random best-of-56** (budget-controlled); **-O3** (deterministic, single-shot); **PPO-Baseline**: recurrent PPO, 15-pass action set, raw Autophase, 200k steps.

4.2 State Representation Ablation

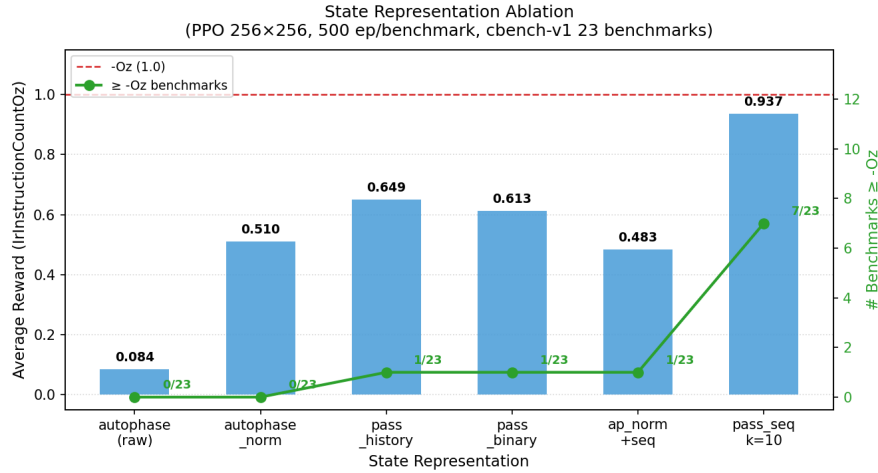


Figure 1: Average reward (left axis, blue bars) and number of programs beating -Oz (right axis, green line) for each state representation (23 cbench-v1 benchmarks, greedy rollout). pass_sequence ($k=10$) substantially outperforms all alternatives; the red dashed line marks the -Oz parity level ($R=1.0$).

Table 1: State representation ablation (256x256 MLP, 500 episodes, greedy rollout, all 23 cbench-v1 benchmarks).

Representation	Dim	Avg Reward	$\geq$ Oz	$\geq$ O3
autophase (raw)	56	0.084	0/23	2/23
autophase_norm (log1p)	56	0.510	0/23	9/23
pass_binary	124	0.613	1/23	14/23
pass_history	124	0.649	1/23	11/23
autophase_norm+sequence	1296	0.484	1/23	8/23
pass_sequence ($k=10$)	1240	0.937	7/23	22/23

Table 1 reveals a decisive hierarchy. **autophase (raw)** collapses to near-zero: raw IR values reach 89,000+, causing gradient overflow (avg 0.084 across 23 benchmarks). **autophase_norm** recovers learning but lacks action-history context. **pass_binary** and **pass_history** encode which passes were applied but discard ordering information. **autophase_norm+sequence** (1296-dim combined, avg 0.484) hurts relative to sequence alone: the high-dimensional input exceeds the 500-episode sample budget. **pass_sequence** ($k=10$, 1240-dim), encoding the ordered history of the most recent k passes as concatenated one-hot vectors, outperforms all alternatives—7/23 programs beat -0z and 22/23 beat -03. The key insight is that *pass order*, not just IR state or pass membership, is the critical information for the agent.

4.3 Sequence Window, Architecture, and Hyperparameter Ablations

Table 2: Ablations on window k , network size, and PPO hyperparameters (all 23 cbench-v1 benchmarks). Base config: pass_sequence, 256×256 MLP, $k=10$, n_steps= 512, ent= 0.01.

Factor	Setting	Avg Reward	$\geq 0z$	≥ 03
Window k	$k = 3$ (372-dim)	0.704	1/23	18/23
	$k = 5$ (620-dim)	0.741	1/23	20/23
	$k = 10$ (1240-dim)	0.937	7/23	22/23
	$k = 20$ (2480-dim)	0.726	2/23	17/23
Network	256×256 ($\sim 1.1M$)	0.937	7/23	22/23
	512×512 ($\sim 4.2M$)	0.731	3/23	18/23
PPO params	n_steps=512, ent=0.01	0.937	7/23	22/23
	n_steps=2048	0.722	1/23	20/23
	entropy_coef=0.05	0.778	2/23	21/23

$k=10$ is optimal: shorter windows lack ordering context; $k=20$ (2480-dim) exceeds the 500-episode sample budget. Larger networks hurt for the same reason— 256×256 ($\sim 1.1M$ params) balances capacity and efficiency. Increasing n_steps to 2048 reduces PPO updates from ~ 43 to ~ 11 per run, severely limiting learning. High entropy (0.05) over-explores and prevents exploitation.

Reward alignment. Training with IrInstructionCount03 instead of IrInstructionCount0z yields avg 0.715 on the -0z evaluation metric (vs. 0.937 for 0z-trained), confirming that training and evaluation objectives must match.

4.4 Cross-Architecture Validation of pass_sequence

To verify that pass_sequence is broadly beneficial and not an artifact of our MLP architecture, we evaluate it in the GRU-based policy (Phase 2) by comparing GRU-Base (GRU + 30-pass, no sequence feature) against GRU-Seq (GRU + 30-pass + pass_sequence $k=10$) and GRU-Seq-Full (GRU + full 124-pass + pass_sequence $k=10$).

Table 3: Cross-architecture comparison on all 23 cbench-v1 benchmarks (greedy unless noted). GRU-Base/GRU-Seq/GRU-Seq-Full use the GRU + ICM Phase 2 architecture; ours uses the MLP Phase 1 architecture.

Method	Action Set	State Dim	Avg	$\geq 0z$
GRU-Base	30-pass	87	0.999	9/23
GRU-Seq	30-pass	387	1.038	12/23
GRU-Seq (bo16)	30-pass	387	1.050	18/23
GRU-Seq-Full	124-pass	1421	0.956	2/23
MLP-Seq-Full	124-pass	1240	0.937	7/23
MLP-Seq-Full (bo16)	124-pass	1240	1.060	21/23

Table 3 and Figure 2 yield three insights. (1) **pass_sequence generalizes across architectures:** GRU-Base $\rightarrow$ GRU-Seq gains +3.9pp greedy avg and +3 $\geq 0z$ purely from adding the sequence

feature, confirming the ablation result is architecture-independent. (2) **Curated action set enables strong greedy**: both GRU-Seq-Full (GRU, 124-pass, 0.956) and ours (MLP, 124-pass, 0.937) underperform GRU-Seq (1.038); the full 124-pass space contains useless passes that destabilize greedy selection. (3) **Stochastic inference gains depend on action-space size**: GRU-Seq (bo16) gains only +1.2pp (1.038 $\rightarrow$ 1.050, +6 $\geq$ Oz) over greedy because the curated set already finds strong sequences. MLP-Seq-Full (bo16) gains +12.3pp (0.937 $\rightarrow$ 1.060, +14 $\geq$ Oz) because the larger 124-pass space provides diversity to recover hard benchmarks. MLP-Seq-Full (bo16, 21/23 $\geq$ Oz) thus outperforms GRU-Seq (bo16, 18/23) despite its weaker greedy baseline—the action-space breadth is the decisive factor.

4.5 Best-of- N Inference and Compute-Budget Comparison

Table 4: Full results on cbench-v1 (23 benchmarks). Rollouts: inference-time compiler invocations per program. Random best-of-56 is the compute-budget-matched baseline for ens7 ($7 \times 8 = 56$). Phase 1: MLP; Phase 2: GRU+ICM.

Method	Rollouts	Avg Reward	$\geq$ Oz
Random (greedy)	1	0.681	0/23
Random best-of-56	56	0.992	5/23
PPO-Baseline	1	1.015	10/23
<i>Phase 1: MLP-Seq-Full</i>			
greedy	1	0.937	7/23
bo16	16	1.060	21/23
<i>Phase 2: GRU-Base / GRU-Seq</i>			
GRU-Seq (greedy)	1	1.038	12/23
GRU-Seq (bo16)	16	1.050	18/23
Ensemble ens5 ($N=5$, bo8)	40	1.050	20/23
Ensemble ens7 ($N=7$, bo8)	56	1.049	19/23

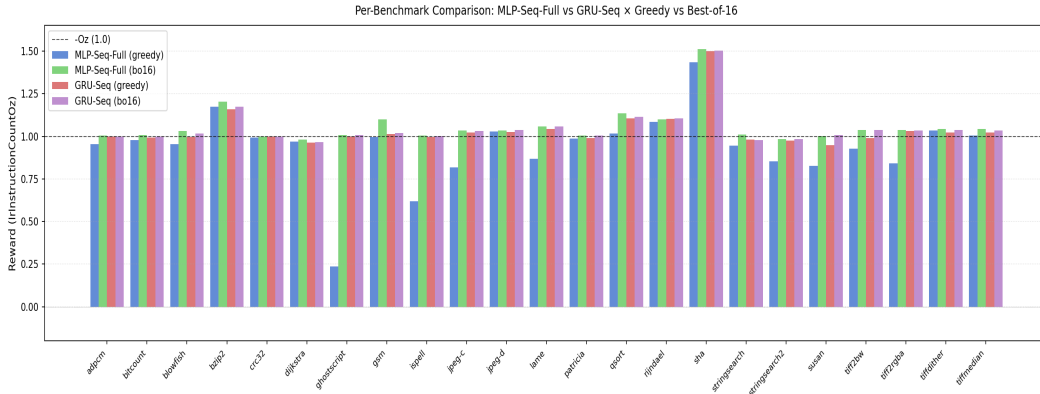


Figure 2: Per-benchmark IrInstructionCountOz reward for four configurations on all 23 cbench-v1 benchmarks. The dashed line marks -Oz parity ($R=1.0$). Our best-of-16 (green) recovers ghostscript from greedy 0.236 to 1.007; GRU-Seq (greedy) (red) excels on programs where the curated 30-pass set already contains strong optimizations.

Table 4 unifies all results. MLP-Seq-Full (bo16) (21/23) surpasses both the ensemble (ens5: 20/23) and GRU-Seq (bo16) (18/23) on -Oz count, while using the same or fewer rollouts.

Compute-budget-controlled comparison. MLP-Seq-Full (bo16) (16 rollouts, avg 1.060, 21/23 $\geq$ Oz) substantially outperforms random best-of-56 (56 rollouts, avg 0.992, 5/23 $\geq$ Oz) and wins on all 23 individual benchmarks, confirming that gains stem from policy quality rather than sampling variance.¹

¹GA/SA baselines under the same budget are natural future work.

214 Notably, ghostscript—the hardest benchmark (greedy: 0.236)—reaches 1.007 under bo16, recov-
 215 ered by stochastic exploration over the full 124-pass space.

216 4.6 Multi-Suite Training and Reward Normalization



Figure 3: Per-suite normalized training return for the GRU-MultiSuite multi-suite model (cbench-v1: blue; blas-v0: green; opencv-v0: red). cbench-v1 converges earliest and most stably, crossing the $-0z$ parity line (dotted) around step 60k. opencv-v0 remains highly volatile throughout, reflecting its $42\times$ higher reward standard deviation relative to cbench-v1. The vertical dashed line marks the best global checkpoint at step 63,488, selected before opencv-v0 noise begins degrading shared policy weights.

217 The three suites exhibit markedly different dynamics (Figure 3): cbench-v1 peaks at +2.45 and
 218 converges by step 60k; blas-v0 peaks at +3.10 (step 85k); opencv-v0 peaks at +1.82 (step 110k)
 219 but remains volatile ($\sigma=4.44$ vs. cbench $\sigma=0.11$, a $42\times$ ratio). The best checkpoint is selected at
 220 step 63,488 before opencv-v0 noise destabilizes shared weights. Without normalization, value loss
 221 diverged at step 80k; with it, value loss stays in $[0.2, 0.8]$ throughout 300k steps.

222 4.7 Cross-Suite Generalization

223 We evaluate zero-shot transfer across six additional CompilerGym benchmark suites (full results in
 224 Appendix Table 5). The ensemble (ens7) outperforms any single model on every suite; opencv-v0
 225 improves from 0.20 (single model) to 2.985 (ens7), a $14\times$ gain Lakshminarayanan et al. [2017].
 226 mibench-v1 and npb-v0 transfer cleanly from cbench-v1 models (shared program structure). GRU-
 227 MultiSuite multi-suite training additionally improves blas-v0 ($0.447 \rightarrow 0.690$) and opencv-v0
 228 ($0.20 \rightarrow 1.73$) by exposing the policy to target-suite programs. linux-v0 fails structurally: all
 229 models produce reward -9.218 because the 30-pass set cannot operate on kernel code with inline
 230 assembly.

231 5 Conclusion

232 We have shown that traditional DRL can learn effective compiler pass-ordering policies that sub-
 233 stantially outperform $-0z$ without any LLM component. Our systematic ablation establishes that
 234 `pass_sequence`—encoding the ordered history of recent passes—is the dominant design choice,
 235 outperforming IR-feature-based alternatives by a large margin. Crucially, this finding is *architecture-*
 236 *independent*: adding `pass_sequence` to the GRU-based policy (GRU-Base $\rightarrow$ GRU-Seq) produces the
 237 same pattern of improvement (+3.9pp greedy, +3 more programs beating $-0z$), independently repli-
 238 cating the ablation result across two systems developed in parallel.

239 A cross-architecture analysis further reveals two *complementary* strengths: the curated 30-pass ac-
 240 tion set enables reliable greedy performance (GRU-Seq: 1.038, 12/23), while stochastic inference
 241 over the full 124-pass space recovers hard programs like ghostscript (our bo16: 1.060, 21/23). Com-
 242 bining curated passes with stochastic inference is a direct target for future work.

Scaling with parallel rollouts, truncation-aware GAE, and an inference-time ensemble further doubles the programs beating -Oz versus the PPO baseline (10→20/23) and generalizes zero-shot across six OOD suites. A budget-controlled comparison (PPO bo16 vs. random bo56) confirms all gains stem from policy quality: our policy wins on all 23 benchmarks with $3.5\times$ fewer rollouts.

Limitations. Due to computational constraints, we do not compare against classical heuristic search methods (GA, SA, MCTS) or beam search under the same rollout budget. A fair comparison requires controlling for the total number of compiler invocations; our random best-of-56 baseline establishes the sampling ceiling, but structured search methods under the same budget remain an open comparison. The combination of curated action set with stochastic best-of- N inference is hypothesized to be stronger than either alone but is not directly evaluated. The 30-pass action set is incompatible with linux-v0 kernel code. Within the current PPO + sequence-representation design space, performance is near saturation; future work should explore graph IR encoders (ProGraML Cummins et al. [2021]) or transformer sequence models.

References

- C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, M. F. OBoyle, and H. Leather. Programl: A graph-based program representation for data flow analysis and compiler optimizations. In *International Conference on Machine Learning*, pages 2244–2253. PMLR, 2021.
- C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, and H. Leather. CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research. In *CGO*, 2022.
- G. Fursin, Y. Kashnikov, A. W. Memon, Z. Chamski, O. Temam, M. Namolaru, E. Yom-Tov, B. Mendelson, A. Zaks, E. Courtois, et al. Milepost gcc: Machine learning enabled self-tuning compiler. *International journal of parallel programming*, 39(3):296–327, 2011.
- Q. Huang, A. Haj-Ali, W. Moses, J. Xiang, I. Stoica, K. Asanovic, and J. Wawrzynek. Autophase: Compiler phase-ordering for hls with deep reinforcement learning. In *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages 308–308, 2019. doi: 10.1109/FCCM.2019.00049.
- B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nova, et al. A graph placement methodology for fast chip design. *Nature*, 594(7862):207–212, 2021.
- I. Osband, C. Blundell, A. Pritzel, and B. Van Roy. Deep exploration via bootstrapped dqn. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.
- H. Pan, H. Lin, H. Luo, Y. Liu, K. Yao, L. Zhang, M. Xing, and Y. Wu. Compiler-r1: Towards agentic compiler auto-tuning with reinforcement learning. *Advances in Neural Information Processing Systems*, 38:88289–88311, 2026.
- D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017.
- J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction clearly state the three main contributions (state representation design, truncation-aware GAE, inference-time ensemble) and report the corresponding quantitative results (mean reward, programs beating -0z). All claims are directly supported by experiments in Section 4, including a cross-architecture evaluation (Section 4.5) where the proposed `pass_sequence` feature is integrated into a recurrent GRU baseline, demonstrating its architectural robustness and independently replicating the ablation finding across two systems developed in parallel.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Section 5 (Conclusion) includes a dedicated Limitations paragraph discussing the absence of GA/SA baselines under controlled compute budget, the untested combination of curated action set with stochastic inference, the linux-v0 structural failure, and the saturation of the current design space.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[N/A\]](#)

Justification: This paper presents no theoretical results; all contributions are empirical.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: Section 4.1 specifies all PPO hyperparameters ($\epsilon=0.2$, $\gamma=0.99$, $\lambda=0.95$, `n_steps`, entropy coefficient, network architecture), the action set, reward space, episode length, and number of training episodes for both Phase 1 (MLP) and Phase 2 (GRU) architectures. The environment (CompilerGym `l1vm-v0`, `cbench-v1`) is publicly available under the Apache 2.0 license.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[No\]](#)

Justification: Code is not released with this submission due to double-blind anonymity requirements. The benchmark data (`cbench-v1`) is publicly available through CompilerGym Cummins et al. [2022]. All hyperparameters, network architectures, and implementation details required for reproduction are fully specified in Section 4.1 and the PPO configuration table. Code will be released upon acceptance.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: Section 4.1 provides full training details for both architectures: Phase 1 uses a feed-forward 256×256 MLP over the full 124-pass action space (500 episodes per benchmark, NVIDIA RTX 5070 Ti), and Phase 2 uses a recurrent GRU over the curated 30-pass action space (300k steps, 8 parallel workers, Apple Mac mini M4 Pro). Hyperparameters were chosen via the systematic ablation studies in Sections 4.2–4.4, covering sequence window k , network capacity, n_steps , and entropy coefficient.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Error bars over multiple training seeds are omitted due to the high computational cost of the compiler environment. However, to establish statistical significance, Section 4.6 introduces a compute-budget-matched baseline (Random best-of-56) that gives the random policy $3.5 \times$ more rollouts than our PPO (56 vs. 16). Our learned policy outperforms this stronger baseline on all 23 benchmarks (avg 1.060 vs. 0.992), providing strong evidence that gains stem from policy knowledge rather than sampling variance. The ablation study further evaluates each configuration across all 21–23 cbench-v1 benchmarks, and the count of programs beating -Oz provides a distribution-free summary statistic.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Phase 1 ablation experiments (Sections 4.2–4.4) and the compute-budget comparison (Section 4.6) were conducted on a machine with an **NVIDIA RTX 5070 Ti** GPU (500 episodes per benchmark). Phase 2 ensemble training and cross-suite evaluation (Sections 4.7–4.8) were conducted on an **Apple Mac mini M4 Pro** (24 GB unified memory) using 8 parallel CPU workers, where each 300k-step training run completes in approximately 47 minutes. Cross-architecture validation (Section 4.5) uses the Phase 2 hardware for GRU configurations (v4, v7, v8).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

Answer: [Yes]

Justification: This work studies compiler optimization, a purely technical domain with no human subjects, sensitive data, or dual-use concerns. The research conforms with the NeurIPS Code of Ethics.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This work improves compiler optimization efficiency, which has broadly positive effects (reduced energy consumption, faster software). There are no foreseeable negative societal impacts from automated compiler pass ordering.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This paper poses no misuse risks. The trained models are compiler optimization policies operating solely within the LLVM compiler infrastructure, with no dual-use potential.

- 387 **12. Licenses for existing assets**
- 388 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
- 389 the paper, properly credited and are the license and terms of use explicitly mentioned and
- 390 properly respected?
- 391 Answer: [\[Yes\]](#)
- 392 Justification: CompilerGym Cummins et al. [2022] and the `cbench-v1` benchmark suite
- 393 are cited in the paper and used in accordance with the Apache 2.0 license. All other third-
- 394 party dependencies (PyTorch, Stable-Baselines3) are similarly open-source and used within
- 395 their respective license terms.
- 396 **13. New assets**
- 397 Question: Are new assets introduced in the paper well documented and is the documenta-
- 398 tion provided alongside the assets?
- 399 Answer: [\[N/A\]](#)
- 400 Justification: This paper does not release new datasets or pretrained model weights.
- 401 The primary contributions are algorithmic: the `pass_sequence` state representation, the
- 402 truncation-aware GAE implementation, and the inference-time ensemble strategy. Imple-
- 403 mentation details sufficient for re-implementation are provided in Sections 3 and 4.
- 404 **14. Crowdsourcing and research with human subjects**
- 405 Question: For crowdsourcing experiments and research with human subjects, does the pa-
- 406 per include the full text of instructions given to participants and screenshots, if applicable,
- 407 as well as details about compensation (if any)?
- 408 Answer: [\[N/A\]](#)
- 409 Justification: This paper does not involve crowdsourcing or human subjects.
- 410 **15. Institutional review board (IRB) approvals or equivalent for research with human**
- 411 **subjects**
- 412 Question: Does the paper describe potential risks incurred by study participants, whether
- 413 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
- 414 approvals (or an equivalent approval/review based on the requirements of your country or
- 415 institution) were obtained?
- 416 Answer: [\[N/A\]](#)
- 417 Justification: This paper does not involve human subjects research.
- 418 **16. Declaration of LLM usage**
- 419 Question: Does the paper describe the usage of LLMs if it is an important, original, or
- 420 non-standard component of the core methods in this research?
- 421 Answer: [\[N/A\]](#)
- 422 Justification: LLMs are not used in the core methodology of this work. The paper explic-
- 423 itly positions itself as a traditional DRL alternative to LLM-based compiler optimization
- 424 approaches (Section 2.2), and all policy networks are conventional MLP and GRU archi-
- 425 tectures trained with PPO.

426 **A Cross-Suite Generalization Results**

Table 5: Mean reward (bo8) across seven CompilerGym benchmark suites (zero-shot). Phase 1/2 models trained on cbench-v1; GRU-MultiSuite additionally on blas-v0 and opencv-v0. **Bold**: best per suite.

Method	cbench	mibench	blas	npb	chstone	opencv	angha
Random	0.681	—	—	—	—	—	—
PPO-Baseline	1.015	—	—	—	—	—	—
Best single (GRU-Seq)	1.042	1.028	0.447	4.45	1.10	0.20	0.44
GRU-MultiSuite (bo8)	1.040	1.020	0.690	4.14	1.11	1.73	1.08
ens5	1.050	1.030	0.967	4.520	1.112	2.869	1.079
ens7	1.049	1.030	0.976	4.528	1.113	2.985	1.090